La Citadelle

Surveiller les couloirs aériens

Les visiteurs sont entrés. Turing les conduit au sommet de la tour de contrôle, d'où l'on voit les drones de la Citadelle tourner autour des remparts.

Pour éviter les accidents, les drones ne volent que dans **trois couloirs aériens** : à **50 m**, **100 m** et **150 m** d'altitude.

Chaque couloir supporte au maximum **5 drones**. Au-delà, c'est la collision.

> « Une anomalie doit être signalée immédiatement, et clairement. Un silence est la pire des réponses. »

Vous allez approfondir la gestion des erreurs :

- `throw new Error(...)` avec un message précis ;
- une erreur qui **remonte** à travers plusieurs fonctions ;
- `try` / `catch` et `error.message`.

Instructions

Complétez les fonctions du fichier `couloirs.js`.

Les altitudes des drones sont fournies sous forme de liste de nombres, par exemple :

```
[50, 100, 50, 150, 50]
```

Le fichier définit déjà la constante :

```
export const MAX_DRONES_PER_CORRIDOR = 5;
```

1. Valider une altitude

Implémentez :

```
validateAltitude(altitude)
```

Règles :

- `50`, `100` ou `150` → retourner `altitude` ;
- toute autre valeur → lancer une erreur dont le message indique l'altitude :

```
validateAltitude(75);
// => lance Error("Altitude interdite : 75 m")
```

Utilisez `switch`.

2. Compter les drones d'un couloir

Implémentez :

```
countAtAltitude(altitudes, altitude)
```

La fonction retourne le nombre de drones qui volent à `altitude`.

```
countAtAltitude([50, 100, 50, 150, 50], 50);
// => 3
```

Utilisez `for...of`.

3. Détecter une collision dans un couloir

Implémentez :

```
checkCorridor(altitudes, altitude)
```

- s'il y a **plus de 5** drones à `altitude`, lancer :

```
// pour altitude = 100
Error("Collision à 100 m")
```

- sinon, retourner le nombre de drones dans ce couloir.

```
checkCorridor([100, 100, 100, 100, 100], 100);
// => 5

checkCorridor([100, 100, 100, 100, 100, 100], 100);
// => lance Error("Collision à 100 m")
```

Utilisez `MAX_DRONES_PER_CORRIDOR` plutôt que d'écrire `5` directement.

4. Contrôler tout l'espace aérien

Implémentez :

```
checkAirspace(altitudes)
```

La fonction doit :

1. valider **chaque** altitude avec `validateAltitude` ;
2. vérifier les trois couloirs (`50`, `100`, `150`) avec `checkCorridor` ;
3. retourner `"Espace aérien sûr"` si aucune erreur n'a été levée.

N'utilisez pas `try` / `catch` ici : si une des fonctions appelées lance une erreur, celle-ci doit **remonter** jusqu'à l'appelant de `checkAirspace`.

```
checkAirspace([50, 50, 100, 150]);
// => "Espace aérien sûr"

checkAirspace([50, 120, 150]);
// => lance Error("Altitude interdite : 120 m")
```

5. Produire un rapport sans planter

Implémentez :

```
airspaceReport(altitudes)
```

La fonction appelle `checkAirspace(altitudes)` :

- si tout va bien, retourner son résultat ;
- si une erreur est levée, retourner `"Alerte : "` suivi du message de l'erreur.

```
airspaceReport([50, 100, 150]);
// => "Espace aérien sûr"

airspaceReport([50, 50, 50, 50, 50, 50]);
// => "Alerte : Collision à 50 m"
```

À rechercher si nécessaire

- `JavaScript throw Error`
- `JavaScript try catch`
- `JavaScript error message`
- `JavaScript for of array`

▶ Aides